Fundamentos de Sistemas Operacionais

Processos e Threads

Prof. Msc. Lucas Reis

lucas.oliveira@puc-campinas.edu.br

Campinas, 2s/2026



Definição de Processo

- Um SO executa um conjunto de programas na perspectiva de um processo
- **Processo** – um programa em execução de maneira sequencial. Não há execução paralela de instruções em um processo.
- Diversas partes
  - Código - **seção texto**
  - Atividade: **contador de programa (PC)**, registradores do processo
  - **Pilha** - dados temporários
  - **Data -** variáveis globais
  - **Heap** - memória alocada dinamicamente





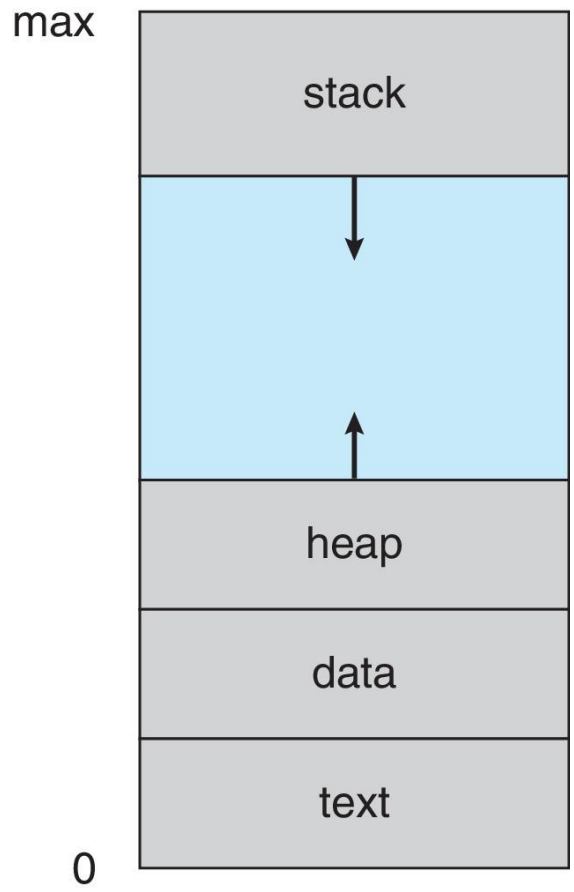
Definição de Processo

- Programa é **passivo** - armazenado em disco (**arquivo executável**); proceso é **ativo**
  - Programa => processo; após carregar seus dados na memória
- Um programa pode ser múltiplos processos
  - Considere múltiplos usuários no mesmo ambiente





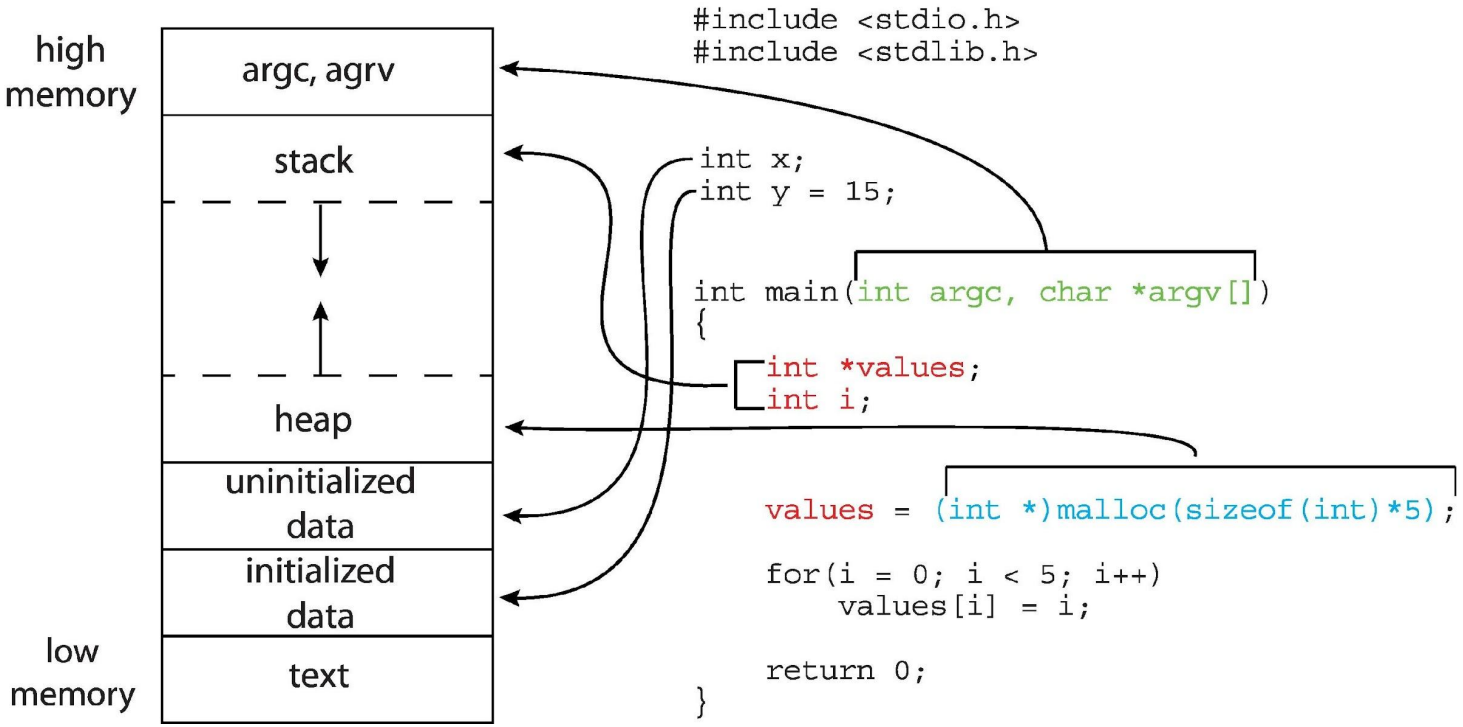
Processo em Memória







Memória de um programa em C







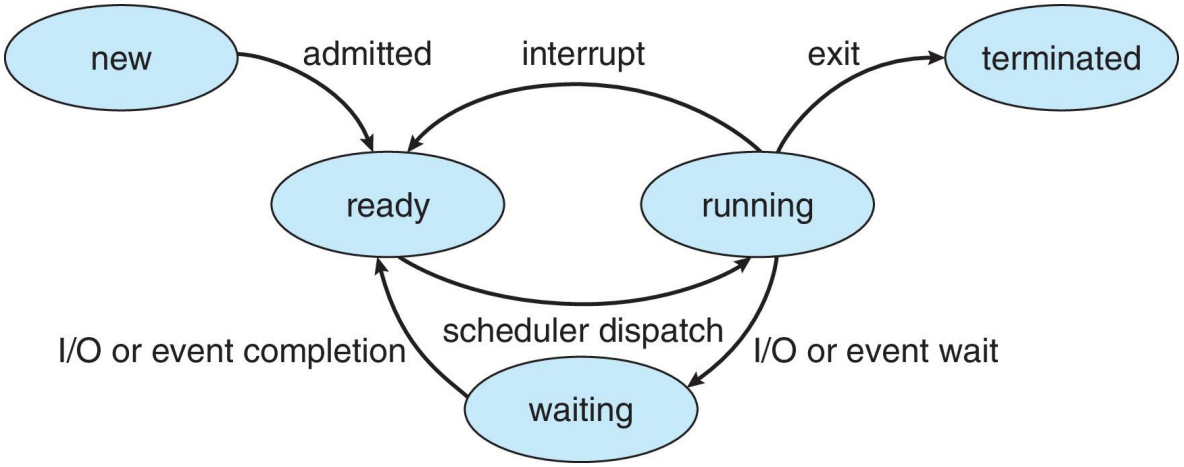
Estado do Processo

- Conforme um processo é executado, seu **estado** é alterado
  - **Novo**: Em criação
  - **Em execução**: Instruções estão sendo executadas
  - **Em espera**: O processo está esperando que algum evento ocorra
    - E/S, sinal, …
  - **Pronto**: Está aguardando atribuição a um processador
  - **Concluído**: O processo terminou sua execução





Diagrama de Estado do Processo



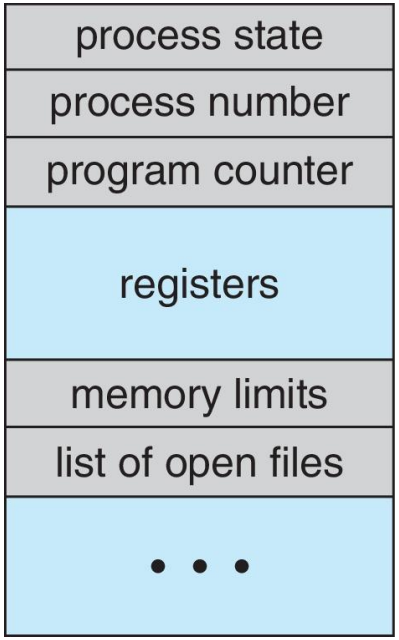




Bloco de Controle do Processo (PCB)

Informações associadas a cada processo

- Estado do processo – executando, esperando, …
- Contador de Programa - próxima instrução a ser executada
- Registradores de CPU - relação de todos os valores a serem alimentados aos registradores
- Informações de escalonamento
- Informações de gerenciamento de memória
- Contadores - métricas de execução (CPU usada, tempo de clock gasto, …)
- Informações E/S – dispositivos alocados ao processo, lista de arquivos abertos







Threads

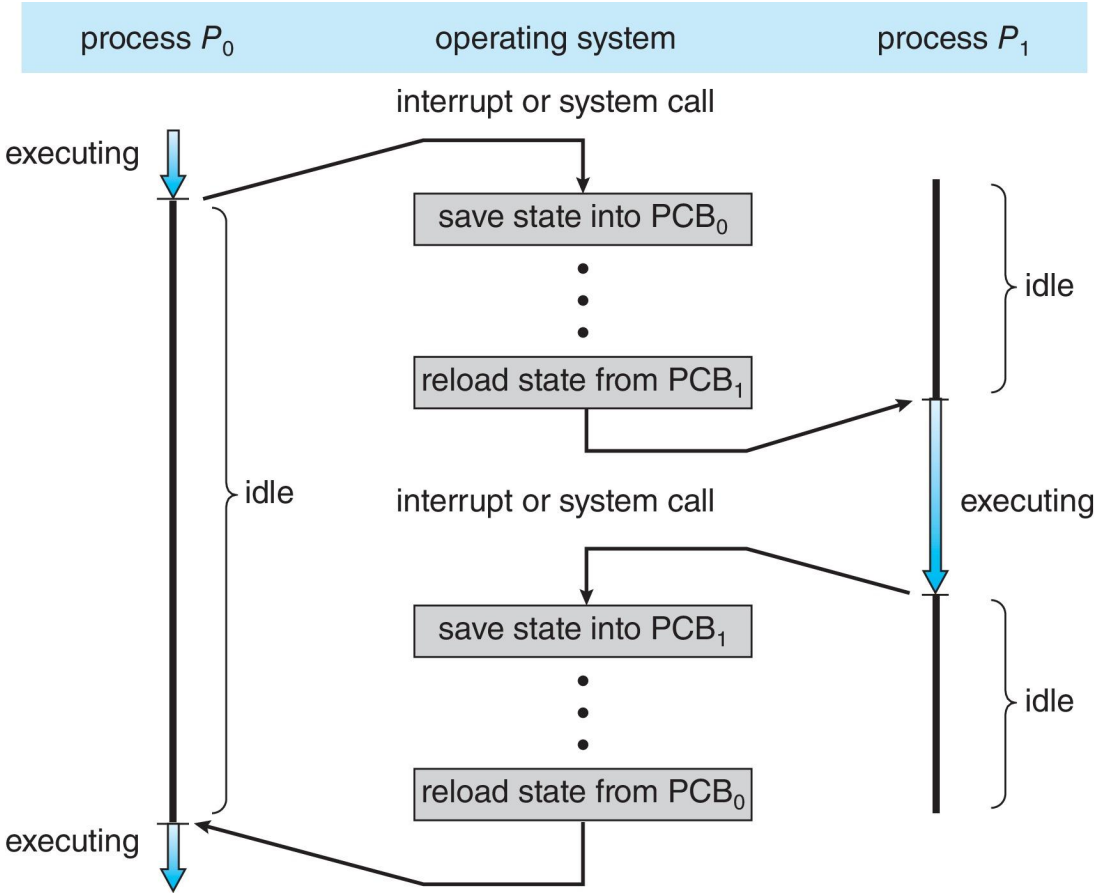
- Até agora, todo processo possui uma *thread* de execução
- Imagine a possibilidade múltiplos contadores de programa por processo
  - Múltiplas instruções seriam executadas ao mesmo tempo
    - Múltiplas threads de controle -> **threads**
- Maior espaço para armazenamento de detalhes, múltiplos **PC** (program counter) no PCB





Troca de Contexto

A **troca de contexto** ocorre quando a CPU troca de um processo para outro







Troca de Contexto

- Quando a CPU troca para outro processo, o sistema deve **salvar o estado** do processo antigo e carregar o **estado salvo** para o novo processo por meio da **troca de contexto**
- **Contexto** de um processo está representado na PCB
- Troca de contexto é puramente gasto de recursos: nenhum trabalho útil é feito durante esse processo
  - Quanto mais complexo o SO e o PCB mais longa a troca de contexto
- Dependente do suporte do hardware
  - algumas arquiteturas possuem muitos registradores na CPU múltiplos contextos carregados de uma vez





Criação de Processos

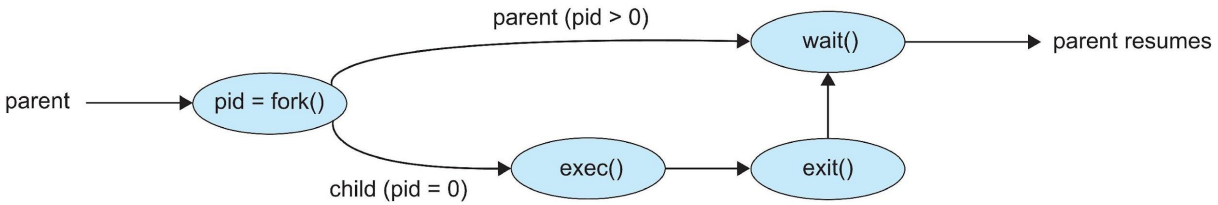
- Processos **pai** criam processos **filhos,** que consequentemente criam outros processos, formando uma **árvore** de processos
- Geralmente, processos são identificados e gerenciados pelo **identificador do processo** (**pid**)
- Opções de compartilhamento de recursos
  - Pai e filho compartilham todos os recursos
  - Filhos compartilham um conjunto dos recursos do pai
  - Pai e filho não compartilham recursos
- Opções de execução
  - Pai e filho executam de maneira concorrente
  - Pai espera os filhos concluírem para executar





Criação de Processos

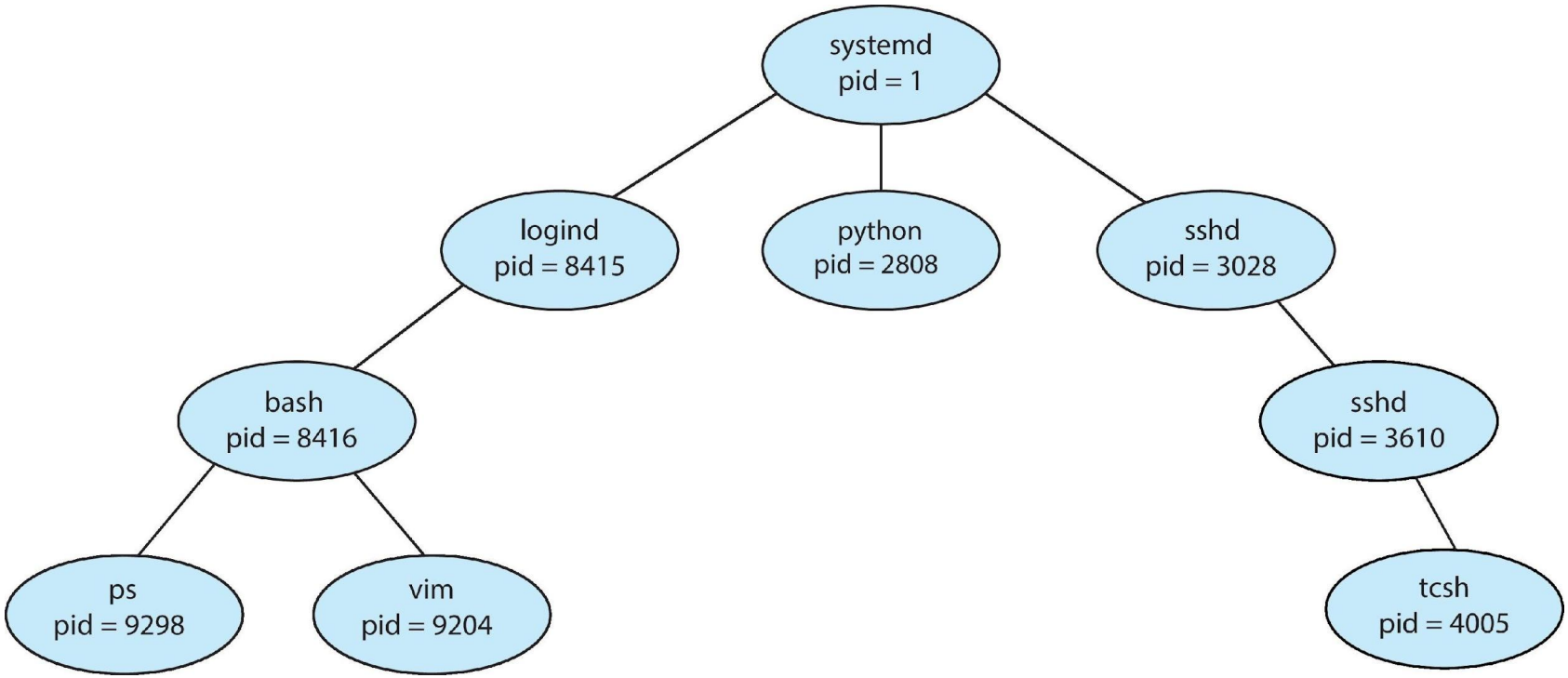
- Espaço de memória (endereços)
  - Filho copia o pai
  - Filho tem um programa carregado no lugar dele
- Exemplos em UNIX
  - chamada de sistema **`fork()`** cria novos processos
  - chamada de sistema **`exec()`** usada após **`fork()`** substitui o espaço de memória do filho por um novo programa
  - Processo pai chama **`wait()`**, esperando o filho concluir







Árvore de processos no Linux







Separação de Processos - Exemplo em C

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```





Terminação do Processo

- Processos executam sua última instrução e pedem ao SO para serem deletados usando a chamada de sistema **`exit()`**
  - Recursos do processo são recuperados pelo SO
- Pais podem encerrar a execução de processos filho usando a chamada de sistema **`abort()`**. Algumas razões:
  - Filho excedeu a quantidade de recursos que poderia alocar
  - Tarefa alocada ao filho não é mais necessária
  - O pai está encerrando, e o SO em questão não permite que filhos coexistam sem um pai





Terminação do Processo

- Alguns SOs não permitem ao filho existir caso após a morte do pai. Se um processo morre, todos os filhos também são removidos
  - **término em cascata.** filhos, netos, …
  - O término é iniciado pelo SO
- O pai pode esperar o término de um processo filho utilizando a chama de sistema **`wait()`**.

```
pid = wait(&status);
```

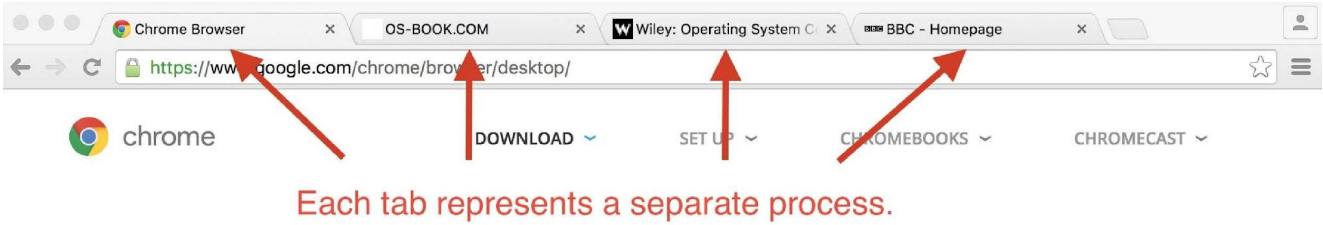
- Se não há nenhum pai esperando (não chamou **`wait()`**), o processo é um **zumbi**
- Se o pai termina sem invocar **`wait()`**, o processo filho se torna **órfão**





Arquitetura Multiprocesso – Chrome Browser

- Muitos navegadores executavam como um único processo
  - Se um web site cria problema, o navegador inteiro trava ou morre
  - Google Chrome (e diversos outros navegadores) usa uma arquitetura multiprocesso com 3 tipos diferentes de processos:
  - **Navegador** gerencia interface de usuário, disco e E/S de rede
  - **Renderizador** processa a renderização de páginas, lida com HTML, Javascript. Um novo renderizador é criado para cada página aberta.
  - **Plug-in** para cada tipo de plug-in







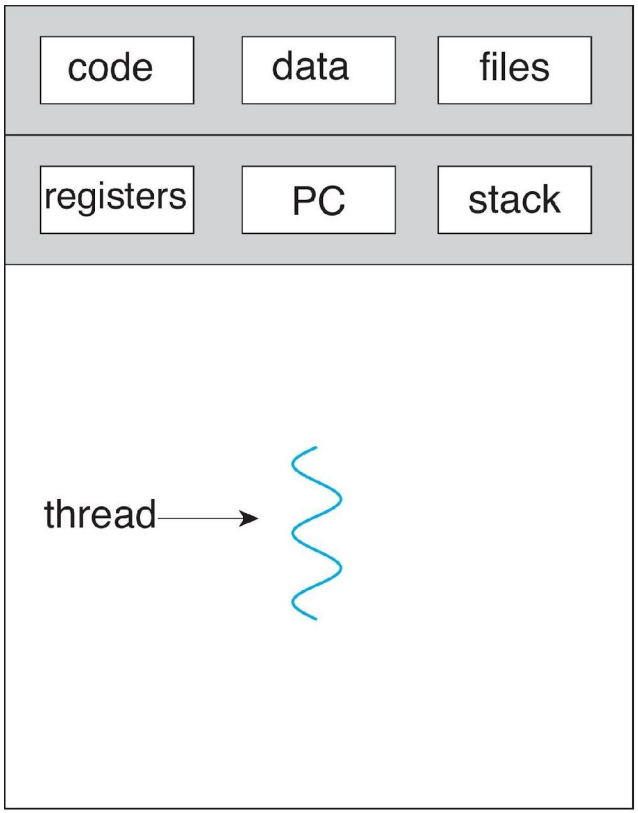
Threads

- Boa parte das aplicações modernas é *multithread*
- Threads executam dentro de uma aplicação
- Múltiplas tarefas dentro da aplicação podem ser implementadas em threads separadas
  - Atualizar a tela
  - Buscar dados
  - Checagem de digitação
  - Responder requisição de rede
- Criar um processo é lento. Criar uma thread é rápido
- Simplifica código, aumenta eficiência
- Kernels são *multithread*

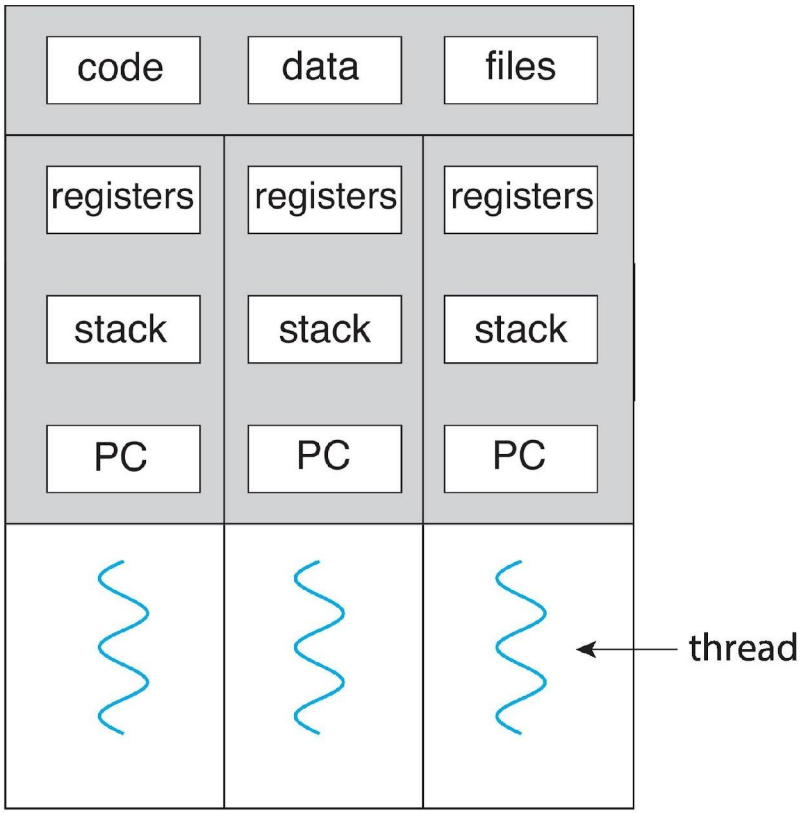




Processos Single e Multi-Thread



single-threaded process



multithreaded process





Benefícios

- **Tempo de resposta –** permite execução se parte do processo está bloqueado (ex. interfaces gráficas)
- **Compartilhamento de recursos –** threads compartilham recursos do processo, mais fácil que memória compartilhada ou troca de mensagens
- **Economia –** mais *"barato"* que criação de processos, menor custo do que troca de contexto
- **Escalabilidade–** processos podem aproveitar a vantagem de arquiteturas multicore





Programação Multicore

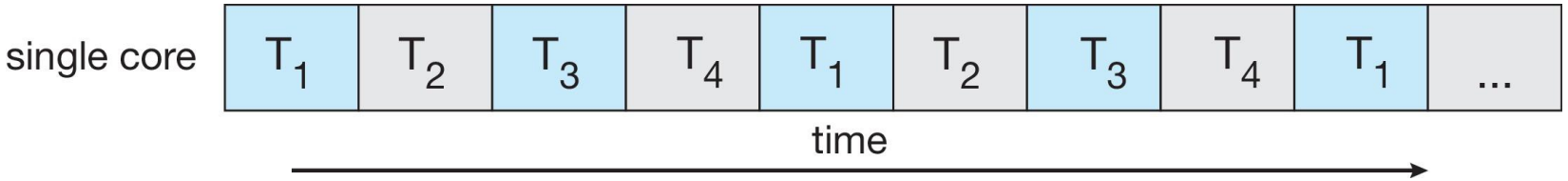
- Sistemas **Multicore** ou **multiprocessadores** pressionam programadores. Desafios incluem:
  - **Dividir atividades**
  - **Equilíbrio de recursos**
  - **Divisão de dados**
  - **Dependência de dados**
  - **Testes e debugging**
- ***Paralelismo*** implica em um sistema que realiza mais de uma tarefa simultaneamente
- ***Concorrência*** suporta mais de uma tarefa progredindo
  - Arquiteturas single core - escalonador provê concorrência, mas não há paralelismo



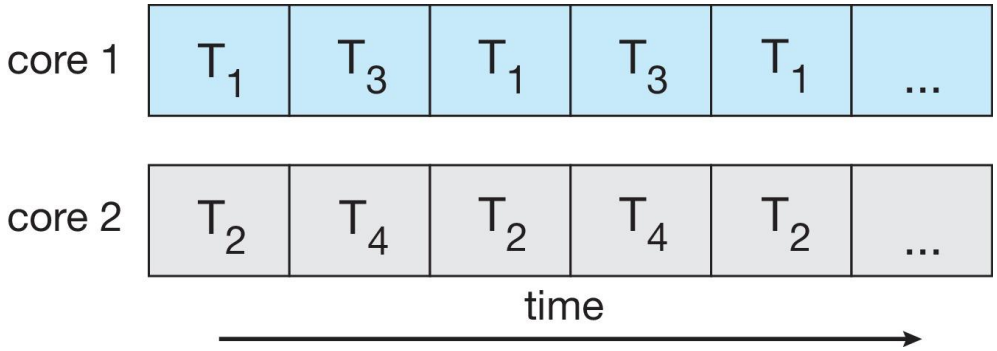


Concorrência e Paralelismo

- **Execução concorrente em sistema single-core:**



- **Paralelismo em sistema multicore:**







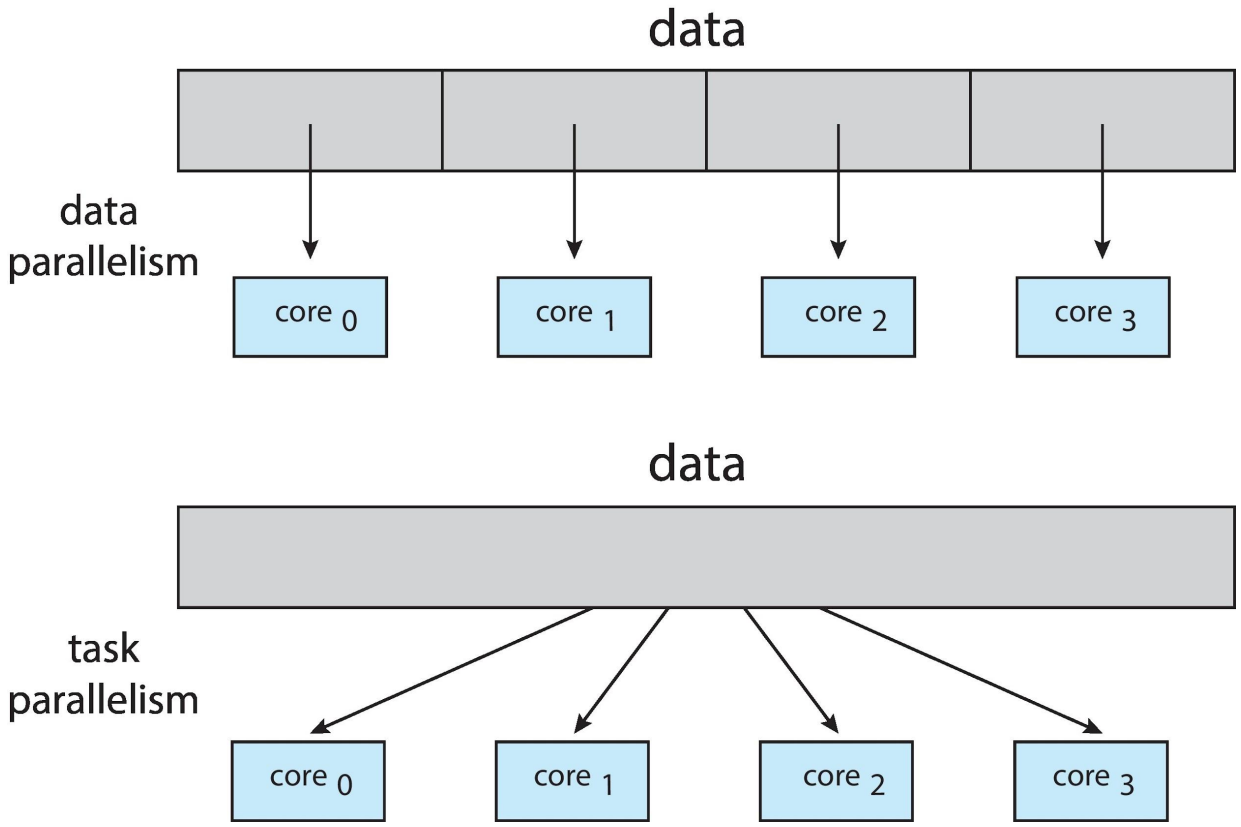
Programação Multicore

- Tipos de paralelismo
  - **Paralelismo de dados** – distribui subconjuntos do mesmo dado entre múltiplos cores, mesma operação em todos
    - Multiplicação de matrizes
    - Treinamento de rede neural
  - **Paralelismo de tarefas** – distribui threads entre cores, cada uma executando uma tarefa diferente
    - Webserver
    - Codificação/decodificação de vídeo





Paralelismo de dados e tarefas







Pthreads

- Pode ser implementado em nível de usuário ou kernel
- Padrão POSIX (IEEE 1003.1c) de API para criação e sincronização de threads
- ***Especificação***, não ***implementação***
- API especifica o comportamento da biblioteca de threads, implementação é responsabilidade do desenvolvedor da biblioteca
- Comum em SOs UNIX (Linux & Mac OS X)





Exemplo Pthreads

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   /* set the default attributes of the thread */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid, &attr, runner, argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```





Exemplo Pthreads

```
/* The thread will execute in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```

